SOFTWARE ENGINEERING

Chapter 8: Requirements Modeling – A Recommended Approach

Based on: Software Engineering: A Practitioner's Approach (9th Edition)

Authors: Roger S. Pressman and Bruce R. Maxim

Student Name: Asal Ghafouri
Instructor: Engineer Rasoulzadeh
Course: Software Engineering

1. Introduction

After requirements have been gathered and understood, they must be transformed into structured models that can guide design and construction. Chapter 8 of Pressman and Maxim's book presents a recommended approach to requirements modeling.

Requirements modeling creates representations of the information domain, the functions the software must perform, and the behavior the software exhibits in response to events. These models help stakeholders and developers share a clear understanding of what must be built.

This chapter explains the objectives of analysis, useful rules of thumb, modeling principles, and the main types of models: scenario-based, class-based, functional, and behavioral. Examples continue with the online shopping application.

2. Requirements Analysis

Requirements analysis is the process of refining and structuring the information obtained during requirements engineering so that it can be used effectively for design.

2.1 Overall Objectives and Philosophy

The main goals of analysis are:

- To obtain a clear understanding of what the customer needs
- To provide a foundation for software design
- To define a set of requirements that can be validated once the software is built

Analysis focuses on the "what" of the system rather than the "how." Implementation details are deliberately postponed.

2.2 Analysis Rules of Thumb

Useful guidelines for analysis include:

1. The model should focus on requirements that are visible in the problem or business domain.
2. The level of abstraction should remain relatively high at the beginning.
3. The analysis model should provide insight into information, function, and behavior.
4. Consideration of infrastructure and non-functional details can often be delayed until later.
5. Keep the model as simple as possible while still providing value to stakeholders.

3. Requirements Modeling Principles

Four fundamental principles guide effective requirements modeling:

1. The information domain of a problem must be represented and understood.
2. The functions that the software performs must be defined.
3. The behavior of the software (as a consequence of external events) must be represented.
4. The models that depict information, function, and behavior must be partitioned in a layered (hierarchical) fashion so that detail is uncovered gradually.

4. Scenario-Based Modeling

Scenario-based models describe the system from the point of view of the people or systems that interact with it (actors).

4.1 Actors and User Profiles

An actor is anything that interacts with the system — a human user, another system, or a device. User profiles help the team understand the goals, skills, and constraints of different kinds of users.

4.2 Creating and Documenting Use Cases

Use cases capture the interactions between actors and the system. A well-written use case typically includes:

- Name and brief description
- Primary and secondary actors
- Preconditions and postconditions
- Main success scenario (basic flow)
- Alternative flows and exception paths

Example – Use Case: Checkout

Primary Actor: Registered Customer

Preconditions: Customer is logged in and the shopping cart contains at least one item.

Main Success Scenario: Customer reviews cart → selects shipping address → chooses payment method → confirms order → system processes payment and creates order → customer receives confirmation.

Exceptions: Payment declined, item out of stock, network failure.

5. Class-Based Modeling

Class-based models represent the system as a set of objects (classes) that have attributes and operations and that collaborate to satisfy requirements.

5.1 Identifying Analysis Classes

Analysis classes are usually derived from:

- Nouns in the problem description or use cases

- Physical or logical entities in the domain
- Events or transactions that must be remembered
- Roles played by people or systems

5.2 Attributes, Operations, and Collaborations

Each class is defined by its attributes (data it holds) and operations (services it provides). Collaborations show how classes work together.

Example:

In the shopping application, candidate classes include Customer, Product, ShoppingCart, Order, and Payment. The ShoppingCart class might have attributes such as items and totalPrice, and operations such as addItem(), removeItem(), and calculateTotal().

5.3 CRC Modeling

Class-Responsibility-Collaborator (CRC) cards are a lightweight technique for exploring class responsibilities and the other classes they collaborate with.

6. Functional and Behavioral Modeling

6.1 Functional Modeling

Functional models focus on the processing that transforms inputs into outputs. Sequence diagrams are commonly used to show how objects interact over time to accomplish a use case.

6.2 Behavioral Modeling

Behavioral models describe how the system responds to events over time. Common representations include:

- State diagrams – show states of an object and transitions caused by events
- Activity diagrams – show the flow of control or data among activities

Example:

An Order object may move through states such as Created, Paid, Shipped, Delivered, and Cancelled. A state diagram makes these transitions and the events that cause them explicit.

7. A Recommended Approach to Requirements Modeling

A practical approach combines the different modeling perspectives:

1. Start with scenario-based models (use cases and user stories) to capture user goals.
2. Derive analysis classes and their relationships from the scenarios.
3. Use sequence or collaboration diagrams to show how classes interact for important use cases.
4. Add state or activity diagrams for objects or processes that have significant behavior over time.
5. Keep models simple and evolve them only as needed.
6. Validate models with stakeholders before moving deeply into design.

8. Practical Example: Modeling the Online Shopping Application

Applying the recommended approach to the shopping application:

8.1 Scenario-Based View

Key use cases are defined: Browse Products, Search Products, Register, Login, Add to Cart, Checkout, Track Order, and Write Review.

8.2 Class-Based View

Main analysis classes identified: Customer, Product, Category, ShoppingCart, CartItem, Order, OrderItem, Payment, and Review. Relationships such as “Customer places Order” and “Order contains OrderItems” are modeled.

8.3 Behavioral View

The Order class is modeled with states (Pending, Paid, Processing, Shipped, Completed, Cancelled). Sequence diagrams are drawn for the Checkout use case to show interactions among Customer, Cart, Order, and Payment objects.

8.4 Outcome

The resulting models give the team a shared, structured understanding of the requirements and provide a solid starting point for architectural and detailed design.

9. Conclusion

Chapter 8 shows how requirements can be transformed into useful models that represent information, function, and behavior. Scenario-based, class-based, functional, and behavioral models complement one another and help both stakeholders and developers understand the system that must be built.

A recommended approach begins with use cases, derives classes and collaborations, and adds behavioral models where they add value. Keeping models simple, focused, and validated reduces the risk of building the wrong system.

In the online shopping application, these modeling techniques turn informal requirements into a clear foundation for design and construction.

10. References

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner’s Approach* (9th ed.). McGraw-Hill Education.

Cockburn, A. (2001). *Writing Effective Use Cases*. Addison-Wesley.